

PROCESAMIENTO DE DATOS BIOMECÁNICOS

PYTHON: GUÍA 2

1. Objetivo

El objetivo de esta tarea es que cada estudiante desarrolle un programa en Python que:

1. cargue la carpeta con los archivos CSV del experimento de marcha,
2. seleccione un archivo individual para el análisis,
3. lea los metadatos y obtenga la frecuencia de muestreo,
4. lea las señales `Angle_X`, `Linear_Acceleration_Z`, `Segmentation_output` y `Sync`,
5. aplique la corrección de signo sobre `Linear_Acceleration_Z`,
6. calcule el tiempo total del registro,
7. detecte la ventana temporal delimitada por la señal `Sync`,
8. cuente pasos mediante transiciones `S3 -> S0` en `Segmentation_output`,
9. estime velocidad de marcha, velocidad media de pasos y longitud media por paso.

2. Descripción del dataset

Los datos corresponden a registros de marcha del protocolo 10MWT (*10 Meter Walking Test*).

- Hay 10 sujetos.
- Cada sujeto tiene 3 trials.
- En total hay 30 archivos CSV de marcha.
- La convención de nombres es:

`SXX_gait_10MWT_YY.csv`

donde:

- `SXX` identifica al sujeto,
- `gait` identifica la tarea de marcha,
- `10MWT` identifica el protocolo,
- `YY` identifica el trial.

3. Ubicación de los archivos

Los archivos del dataset se encuentran en:

```
data/raw/gait
```

El archivo template para comenzar el desarrollo debe encontrarse en:

```
dataAnalysis/gait/tarea2_template.py
```

4. Formato de cada archivo CSV

Cada archivo CSV tiene dos bloques:

1. Un bloque inicial de metadatos.
2. Un bloque numérico de señales.

Ambos bloques están separados por una línea vacía.

4.1. Bloque de metadatos

El bloque inicial contiene una línea por metadato, por ejemplo:

```
Subject,S01  
Sampling Frequency,62.5
```

En este bloque aparecen campos útiles como:

- Identificación del sujeto.
- Descripción clínica.
- Frecuencia de muestreo.
- Número de muestras.
- Fecha y descripción del trial.

4.2. Bloque de señales

Luego de la línea vacía aparece la tabla numérica con nombres de columnas. Para esta tarea interesan, como mínimo, las siguientes columnas:

- Angle_X
- Linear_Acceleration_Z
- Segmentation_output
- Sync

5. Ventana útil de marcha y métricas

En el protocolo 10MWT se considera una ventana útil de 6 metros delimitada por la señal Sync. En esta tarea se utilizará esa ventana para calcular las métricas principales.

5.1. Tiempo total del registro

Si el archivo contiene N muestras y la frecuencia de muestreo es f_s , entonces:

$$T = \frac{N}{f_s}$$

5.2. Velocidad de marcha

Si entre la primera y la última activación de Sync hay N_{sync} muestras, entonces el tiempo de la ventana útil es:

$$T_{sync} = \frac{N_{sync}}{f_s}$$

y la velocidad media de marcha se estima como:

$$v_{marcha} = \frac{6}{T_{sync}} = \frac{6 \cdot f_s}{N_{sync}}$$

5.3. Conteo de pasos

Para este ejercicio se considerará un paso cada vez que la señal Segmentation_output haga una transición:

$$3 \rightarrow 0$$

dentro de la ventana delimitada por Sync.

5.4. Velocidad media de pasos

Si dentro de la ventana útil se detectan N_{pasos} pasos, entonces:

$$v_{pasos} = \frac{N_{pasos}}{T_{sync}}$$

5.5. Longitud media por paso

Una estimación sencilla de la distancia media recorrida por paso es:

$$L_{paso} = \frac{v_{marcha}}{v_{pasos}}$$

6. Tareas a realizar

Cada estudiante debe completar el archivo `tarea2_template.py` implementando las funciones principales del flujo de trabajo.

6.1. Carga de ficheros y selección del registro

Completar la lógica necesaria para:

- listar los archivos CSV de la carpeta,
- mostrar un resumen de la carpeta cargada,
- seleccionar un archivo usando su índice en la lista,
- construir una estructura tipo `RegistroCSV` con la información del archivo elegido.

La idea es mantener el mismo esquema de entrada utilizado en la Guía 1, pero concentrando el análisis posterior en un solo registro.

6.2. Carga de metadatos

Implementar la lectura del bloque inicial del CSV para guardar:

- El nombre del campo.
- El valor asociado.

Los metadatos deben cargarse en el atributo `metadatos` del registro.

6.3. Carga de señales

Implementar la lectura del bloque numérico del CSV y conservar solo las columnas necesarias para la tarea:

- `Angle_X`.
- `Linear_Acceleration_Z`.
- `Segmentation_output`.
- `Sync`.

Estas señales deben almacenarse en el atributo `datos` del registro.

6.4. Frecuencia de muestreo

Implementar la función que busque en los metadatos el campo:

`Sampling Frequency`

y devuelva ese valor como número en punto flotante.

6.5. Corrección de aceleración

Implementar una función que cambie el signo de la columna:

`Linear_Acceleration_Z`

para todas las muestras validas del registro.

6.6. Detección de ventana Sync

Implementar funciones que:

- encuentren el índice de la primera muestra con `Sync != 0`,
- encuentren el índice de la última muestra con `Sync != 0`,
- calculen cuantas muestras quedan comprendidas entre ambas.

6.7. Conteo de pasos

Implementar una función que recorra `Segmentation_output` dentro de la ventana Sync y cuente cuántas transiciones 3 $\rightarrow$ 0 aparecen.

6.8. Cálculo de métricas

Implementar las funciones necesarias para obtener:

- tiempo total del registro,
- velocidad media de marcha,
- velocidad media de pasos,
- longitud media estimada por paso.

7. Template de trabajo

El archivo `tarea2_template.py` incluye:

- La definición de la estructura `RegistroCSV`.
- El flujo principal en `main()`.
- Funciones auxiliares ya resueltas para listar la carpeta, seleccionar un archivo y manejar líneas del CSV.
- Las funciones principales declaradas con comentarios que indican qué debe hacer cada parte.

El estudiante debe completar los cuerpos de las funciones sin cambiar innecesariamente la estructura general del script.

8. Entrega

La entrega consiste en subir el código desarrollado al repositorio de trabajo del curso.

1. Deben compartir la dirección (enlace, link) de su repositorio en GitHub al hacer la entrega del trabajo en el aula virtual.
2. Deben utilizar el repositorio para dejar constancia de la colaboración entre los miembros del grupo.
3. La carpeta de trabajo en el repositorio debe ser

`dataAnalysis/gait/`

4. El archivo principal a completar es `dataAnalysis/gait/tarea2_template.py`, y deben guardarlo como `Tarea2.py`. Recuerde, todos los archivos deben estar en el repositorio.
5. Los archivos `.py` deben estar documentados y se debe indicar el/los autor/es del código en el encabezado comentado.
6. Si el estudiante crea archivos auxiliares, también deben quedar versionados en el repositorio.
7. El código entregado debe ejecutar sin errores dentro del proyecto.

9. Resultado esperado

Al finalizar la tarea, el programa debería ser capaz de:

1. Cargar la carpeta de archivos CSV de marcha y seleccionar uno de ellos.
2. Separar correctamente metadatos y señales del archivo elegido.
3. Obtener la frecuencia de muestreo desde el encabezado.
4. Corregir el signo de `Linear_Acceleration_Z`.
5. Detectar la ventana temporal marcada por `Sync`.
6. Contar pasos mediante transiciones `S3 ->S0`.
7. Estimar tiempo total, velocidad de marcha, velocidad de pasos y longitud media por paso.